full end-to-end scenario with a web browser, web server, our application, and a database.

Let's run the existing system tests using `bin/rails test:system`. Oh dear, there are about a dozen failures—which isn't all that surprising, given that we've ignored these tests up to this point.

The output indicates screenshot images have been placed into the `/tmp/screenshots` directory, and taking a look at a few of them, we feel a bit like archaeologists. The tests verify the operation of the code as originally scaffolded—most importantly before we added product validation logic in *Iteration B1: Validating!*, and before we moved the cart in*Changing the Flow*.

We could fix these errors, but we would end up with tests that largely duplicate tests we already have. Let's clean things up and write an entirely new test—one that takes advantage of the fact that we're interacting with a real browser that runs the JavaScript code that we provided.

```
$ rm test/system/carts_test.rb
$ rm test/system/line_items_test.rb
$ rm test/system/products_test.rb
```

Now we're ready to write the test we came here to write, which is that our JavaScript is working when it's run in a web browser. We start by describing the actions and checks we want performed in `test/system/orders_test.rb`, which already has some tests in it from the scaffold:

rails80/depot_p/test/system/orders_test.rb

```
require "application_system_test_case"

class OrdersTest < ApplicationSystemTestCase
  test "check dynamic fields" do
    visit store_index_url

    click_on "Add to Cart", match: :first
```